# SNK ARCHITECT

FINAL MASTER DEVELOPMENT PROMPT

## Design Beyond Imagination.

A practical build brief for a premium architecture website with an interactive Three.js house and interior visualization studio.

Technology: HTML • CSS • JavaScript • Three.js

### How to use

Upload this PDF to Claude or copy the prompt text from the document. Ask Claude to deliver the files in numbered batches if the response is too long. Test the website locally before publishing it on GitHub Pages.

*Important: This document is a development specification. Features must be implemented and tested; they are not automatically present just because they are listed here.*

# MASTER PROMPT

Copy the prompt below into Claude

```
PROJECT: SNK Architect
TAGLINE: Design Beyond Imagination.

ROLE
Act as a senior Three.js developer, architectural visualization specialist, UI/UX designer, and frontend
  engineer.

MISSION
Build a professional, responsive, low-cost architectural visualization website with an interactive 3D
  house and interior design studio.

TECHNOLOGY
- HTML5
- CSS3
- Vanilla JavaScript using ES modules
- Three.js
- OrbitControls
- GLTFLoader for future GLB/GLTF models
- GitHub Pages-compatible deployment
- No paid backend or AI API required for Phase 1

BRAND DESIGN
- Premium architectural studio aesthetic
- Deep charcoal: #111820
- Slate: #202A34
- Champagne gold: #D7B77B
- Warm white: #F7F5F0
- Cool grey: #B8C0C8
- Strong typography, clean spacing, refined layouts
- Brand name: SNK Architect
- Tagline: Design Beyond Imagination.
- Supporting line: Design • Build • Visualize

PHASE 1 — WEBSITE
Create:
1. Professional homepage with hero section.
2. Services section.
3. Architecture and interior portfolio gallery.
4. Interactive 3D Design Studio.
5. 2D–6D workflow explanation.
6. About section.
7. Contact and project inquiry form.
8. Responsive mobile navigation.
9. Professional footer and social links.
10. Clear calls to action.

PHASE 2 — INTERACTIVE THREE.JS STUDIO
Build a functional 3D room scene containing:
- Floor, walls, ceiling and windows.
- Door and basic furniture.
- Physically based materials where appropriate.
- Realistic scale and room proportions.
- Perspective camera and OrbitControls.
- Smooth camera orbit, zoom and pan.
- Reset camera button.
- Daylight and night lighting presets.
- Wall color selector.
- Floor material/color selector.
- Toggleable furniture visibility.
- Fullscreen viewing where supported.
- Responsive canvas resizing.
- Loading and error states.
Use real 3D geometry and materials, not a static image pretending to be a 3D model.
```

PHASE 3 — ARCHITECTURAL VISUAL QUALITY
- Use high-quality textures where available and properly licensed.
- Configure realistic lighting and shadows.
- Use appropriate roughness, metalness, normal maps and UVs.
- Use tone mapping and color management appropriately.
- Optimize texture sizes and polygon counts for mobile devices.
- Support loading external GLB/GLTF models.
- Include a quality/performance option if practical.
- Never claim real-time graphics are equivalent to a professional offline render.

PHASE 4 — PROJECT PRESENTATION
- Portfolio cards for living rooms, bedrooms, kitchens and house exteriors.
- Project details and material information.
- Before/after concept comparison.
- Export or download a screenshot of the current 3D view when supported.
- Project inquiry form with validation and clear submission feedback.

PHASE 5 — FUTURE EXTENSIONS
Prepare the architecture for future additions:
- AI room redesign using an external image-generation API.
- User image uploads.
- House exterior visualization.
- Furniture and material libraries.
- Project saving and sharing.
- Budget estimates based on editable, clearly labelled rates.
Do not pretend these future features are already functional.

CODE STRUCTURE
Create separate files for:
- index.html
- services.html
- portfolio.html
- contact.html
- css/style.css
- css/responsive.css
- css/studio.css
- js/main.js
- js/scene.js
- js/camera.js
- js/materials.js
- js/lighting.js
- js/furniture.js
- js/controls.js
- js/export.js
- README.md

ENGINEERING REQUIREMENTS
- Every visible button must work or be clearly marked as unavailable.
- Avoid placeholder links and fake functionality.
- Use accessible labels and keyboard-friendly controls.
- Handle loading errors and missing assets gracefully.
- Do not expose API keys or secrets in frontend code.
- Avoid unnecessary dependencies.
- Use a consistent, maintainable coding style.
- Explain all external asset requirements.
- Ensure relative asset paths work on GitHub Pages project URLs.
- Do not use a backend unless explicitly required.
- Use Three.js for all custom 3D coding.

DELIVERABLES
1. Complete source code for every required file.
2. Exact folder structure.
3. Setup and local-running instructions.
4. GitHub Pages deployment instructions.
5. A checklist for testing on desktop and mobile.
6. A list of known limitations.
7. Troubleshooting steps for common errors.

WORKING METHOD
First inspect the requirements and propose the implementation structure.
Then build the first working version.
Test the core 3D interactions and fix obvious errors.

```
Do not stop after producing a visual mockup.
Do not omit files or replace code with phrases such as "same as above".
If the response becomes too long, deliver the files in clearly numbered batches and track which files are
   complete.

Start with a working Phase 1 prototype. Prioritize functional 3D interaction, visual quality, mobile
  performance and maintainable code over unnecessary features.
```